



Department of Computer Science and Engineering
Midterm Examination Fall 2025
CSE 321: Operating Systems

Duration: 1 Hour 30 Minutes

Total Marks: 30

Answer all the following questions.
 Figures in the right margin indicate marks.

Name:	ID:	Sec:
-------	-----	------

1. a) Write the answer in the question paper. CA-E [1]

CO1 Suppose you are running a background data processing program that spawns a child process to handle file uploads. Suddenly, the parent process crashes before finishing and never calls **wait()** to check the child's exit status.

Write what happens to the child process in this situation?

Answer:

b) Write the outputs in the question paper. The output sequence should be exactly matched. CA-M [1*3 =3]

Write the outputs of each print statement in one line.

```
main:
  int p = 2, q = 3, r = 4
  pid1 = fork()
  if (pid1 == 0):
    p = p + 3
    q = q + 1
    r = r + 2
    print("p:", p, " q:", q, " r:", r)
  else:
    wait(NULL)
    p = p + 4
    q = q + 5
    r = r + 6
    pid2 = fork()
    if (pid2 == 0):
      p = p + 1
      q = q + 1
      r = r + 1
      print("p:", p, " q:", q, " r:", r)
    else:
      wait(NULL)
      print("p:", p, " q:", q, " r:", r)
```

Outputs	
I	
II	
III	

c) Write the answers in the question paper.

CA-E [1*2
=2]

Adhara wants to build a media editing application where multiple processes such as audio, video and effects processes work on the same data simultaneously. Since the data needs to get accessed and updated frequently, she needs a communication method that ensures high speed with minimal kernel support.

- I. **Identify** the suitable interprocess communication method for Adhara in this scenario.
- II. **State** the reason behind the selection.

Answer:

I	
II	

2. a) Write the answer in the answer script.

CA-M [2]

CO1 Dipu has implemented two identical MLFQ schedulers. In the first implementation, new processes arrive at the queue having the highest priority level. In the second implementation, new processes arrive at the queue having the lowest priority level. Apart from the mentioned difference, all other operations of MLFQ are the same as regular MLFQs. A balanced number of I/O and CPU-bound processes arrive in the same order in both implementations of Dipu.

Explain how the different scheduling criteria would differ across both implementations.

b) Write the answers in the answer script.**CA-E**

In an OS, **Multilevel Feedback Queue** functions as a scheduler. In the scheduler, the ready queue is divided into **three** levels. At every level, the round-robin algorithm has been employed, and certain constraints must be maintained, as listed below.

- **Q0:** time quantum = **4**, priority = **0**
- **Q1:** time quantum = **6**, priority = **1**
- **Q2:** time quantum = **8**, priority = **2**

Here, the lower priority value indicates a higher priority, and in the case of an I/O request, the scheduler will promote a process by one level upon completion of the I/O operation. The information on burst times, arrival times, and I/O times of three processes is given below:

Process	Burst Time	Arrival Time	I/O Time
P1	10	4	3s I/O [after total 8s CPU allocation]
P2	11	0	N/A
P3	11	6	5s I/O [after total 9s CPU allocation]

- I. **Draw** a Gantt chart using the multilevel feedback queue scheduling algorithm, showing the states of the ready queues of different levels.
- II. **Calculate** the average waiting time and average turnaround time from the Gantt chart.

[5]**[1+1
=2]**

c) Write the answer in the question paper.

CA-D [2]

You have designed an MLFQ with three total queues, the highest-priority queue being set with a time quantum of **50 ms**, and the next two queues having time quanta of **200 ms** and **500 ms**, respectively. Now you are deciding on the promotion factors of the processes.

You have two options in mind:

- If a process completes an I/O operation, it returns to the same level from which it came.
- It will get promoted to a higher priority level upon completing an I/O operation.

You have an interactive program that repeatedly performs the following from the very beginning after entering the queue: a short burst (**30 ms**) of CPU usage, and then waits for I/O to retrieve user information.

Explain briefly with proper reasons whether there will be any differences in performance between the two choices for this process.

Answer:

3. a) Write the answer in the question paper.

CA-M [2]

CO2 A programmer attempts to use a semaphore to protect a shared counter **x**, as shown below:

```
semaphore mutex = 1;
int x = 0;
```

```
// Thread 1
int temp = x;
wait(mutex);
x = temp + 1;
signal(mutex);
```

```
// Thread 2
int temp = x;
wait(mutex);
x = temp + 1;
signal(mutex);
```

Identify the synchronization mistake in this code.

Answer:

b) Write the answer in the question paper.

CA-E [1]

Explain whether it is always possible or not in a multicore system during multithreading to speed up a program by n times if we use n threads instead of one.

Answer:

c) Write the answer in the answer script.

CA-D [2]

Write the issues with proper logic that might arise if we use recursive functions in a multi-threaded program.

d) Write the answer in the answer script.

CA-D

The variable **counter** is a global variable that is shared by both **thread-1** and **thread-2**. The initial value of **counter** is 0, **temp1** and **temp2** are local variables. Consider **thread-1** and **thread-2** are run at the same time.

Code run by thread-1	Code run by thread-2
<pre>// unrelated statements temp1 = counter; temp1 += 1; counter = temp1; // unrelated statements</pre>	<pre>// unrelated statements temp2 = counter; temp2 += 2; counter = temp2; // unrelated statements</pre>

- I. **Identify** what kind of problem can arise in this scenario and how it can be avoided.
- II. **Modify** the codes of both **thread-1** and **thread-2** above by utilizing **two binary semaphores** to ensure that the final value of **counter** is exactly 2, after both threads have finished their execution. You have to mention the initial values of both semaphores.

[2]

**[1.5+
1.5=
3]**

- e) Write the outputs in the question paper. The output sequence should be exactly matched. CA-M [1*3=3]

```

count = 1
mutex m
semaphore step1
semaphore step2

function worker1(arg1):
    semaphore_wait(step1)
    mutex_lock(m)
    count = count * 2
    print("Thread 1 value ", count)
    mutex_unlock(m)
    semaphore_signal(step2)

function worker2(arg2):
    semaphore_wait(step2)
    mutex_lock(m)
    count = count + 3
    print("Thread 2 value ", count)
    mutex_unlock(m)
    semaphore_signal(step2)

function worker3(arg3):
    mutex_lock(m)
    count = count * 1
    print("Thread 3 value ", count)
    mutex_unlock(m)
    semaphore_signal(step1)

main:
    mutex_initialize(m, NULL)
    semaphore_initialize(step1, 0, 0)
    semaphore_initialize(step2, 0, 0)
    thread_create(t1, NULL, worker1, NULL)
    thread_create(t2, NULL, worker2, NULL)
    thread_create(t3, NULL, worker3, NULL)
    thread_join(t1, NULL)
    thread_join(t2, NULL)
    thread_join(t3, NULL)
    mutex_destroy(m)
    semaphore_destroy(step1)
    semaphore_destroy(step2)

```

Outputs	
I	
II	
III	